# Exponential Smoothing

Guibril Ramde

## 8.8 Exercises

8. **1 Consider the number of pigs slaughtered in Victoria, available in the `aus_livestock` dataset.**

   a. **Use the `ETS()` function to estimate the equivalent model for simple exponential smoothing. Find the optimal values of and $0\ 0$, and generate forecasts for the next four months.**

```
library(fpp3)
```

```
Warning: package 'fpp3' was built under R version 4.5.2
```

```
-- Attaching packages ---------------------------------------- fpp3 1.0.3 --
```

```
v tibble      3.3.1     v tsibble     1.2.0
v dplyr       1.2.0     v tsibbledata 0.4.1
v tidyr       1.3.2     v ggtime      1.0.0
v lubridate   1.9.4     v feasts      0.5.0
v ggplot2     4.0.2     v fable       0.5.0
```

```
Warning: package 'tibble' was built under R version 4.5.2
```

```
Warning: package 'dplyr' was built under R version 4.5.2
```

```
Warning: package 'tidyr' was built under R version 4.5.2
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
Warning: package 'tsibble' was built under R version 4.5.2

Warning: package 'ggtime' was built under R version 4.5.2

Warning: package 'feasts' was built under R version 4.5.2

Warning: package 'fabletools' was built under R version 4.5.2

Warning: package 'fable' was built under R version 4.5.2

-- Conflicts ------------------------------------------------- fpp3_conflicts --
x lubridate::date()    masks base::date()
x dplyr::filter()      masks stats::filter()
x tsibble::intersect() masks base::intersect()
x tsibble::interval()  masks lubridate::interval()
x dplyr::lag()         masks stats::lag()
x tsibble::setdiff()   masks base::setdiff()
x tsibble::union()     masks base::union()
```

```
library(tidyverse)
```

```
Warning: package 'readr' was built under R version 4.5.2

Warning: package 'purrr' was built under R version 4.5.2

-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats 1.0.1     v readr   2.1.6
v purrr   1.2.1     v stringr 1.6.0

-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter()     masks stats::filter()
x tsibble::interval() masks lubridate::interval()
x dplyr::lag()        masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```r
pig_slaughtered <- aus_livestock |>
     filter(State == "Victoria", Animal == "Pigs")

pig_model <- pig_slaughtered |>
   model(ETS(Count ~ error('A') + trend("N") + season("N")))

report(pig_model)
```

```
Series: Count
Model: ETS(A,N,N)
  Smoothing parameters:
    alpha = 0.3221247

  Initial states:
     l[0]
 100646.6

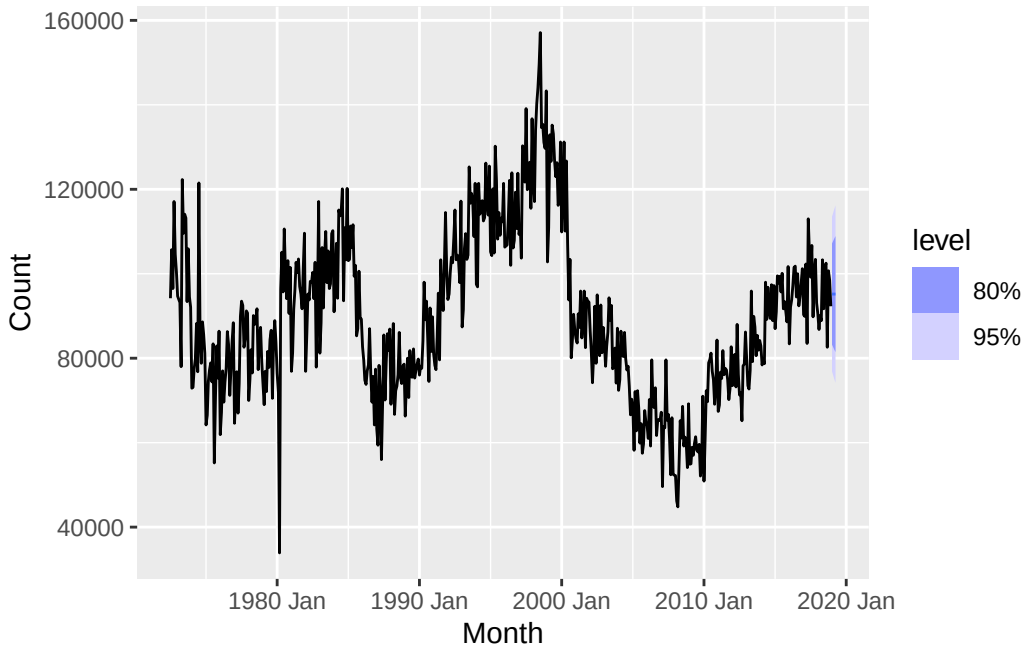
  sigma^2:  87480760

     AIC     AICc      BIC
13737.10 13737.14 13750.07
```

```r
#Forcast
fc_p <- pig_model |>
  forecast(h = 4)
fc_p
```

```
# A fable: 4 x 6 [1M]
# Key:     Animal, State, .model [1]
  Animal State    .model                                                Month
  <fct>  <fct>    <chr>                                                 <mth>
1 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Jan
2 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Feb
3 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Mar
4 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Apr
# i 2 more variables: Count <dist>, .mean <dbl>
```

```r
#plot
fc_p |> autoplot(pig_slaughtered)
```




```
## distinct(Animal)
```

**##Interpretation:**

The optimal smoothing parameter is = 0.3221247, and the estimated initial level is = 100,646.6. The alpha value indicates that the model gives approximately 32.21% weight to the current observation and 67.79% to the previous estimated level. Since the model uses simple exponential smoothing without trend or seasonality, the point forecasts for the next four months remain constant.

**b. Compute a 95% prediction interval for the first forecast using** $y^{\pm 1.96sy} \pm 1.96s$ **where** ss **is the standard deviation of the residuals. Compare your interval with the interval produced by R.**

```
#Extracting then first forecast
first_forcast <- fc_p |>
  as_tibble() |>
  pull(.mean) |>
  first()
# Calculate standard deviation of residuals

s <- pig_model |>
  augment() |>
```

```
  pull(.innov) |>
  sd(na.rm = TRUE)

#Calculate 95% prediction interval
lower <- first_forcast - 1.96 * s
upper <- first_forcast + 1.96 * s

# Results
lower
```

```
[1] 76871.01
```

```
upper
```

```
[1] 113502.1
```

```
fc_p |>
  hilo( level = 95) |>
  as_tibble()
```

```
# A tibble: 4 x 7
  Animal State    .model                                                Month
  <fct>  <fct>    <chr>                                                 <mth>
1 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Jan
2 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Feb
3 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Mar
4 Pigs   Victoria "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"~ 2019 Apr
# i 3 more variables: Count <dist>, .mean <dbl>, `95%` <hilo>
```

**##Interpretation:**

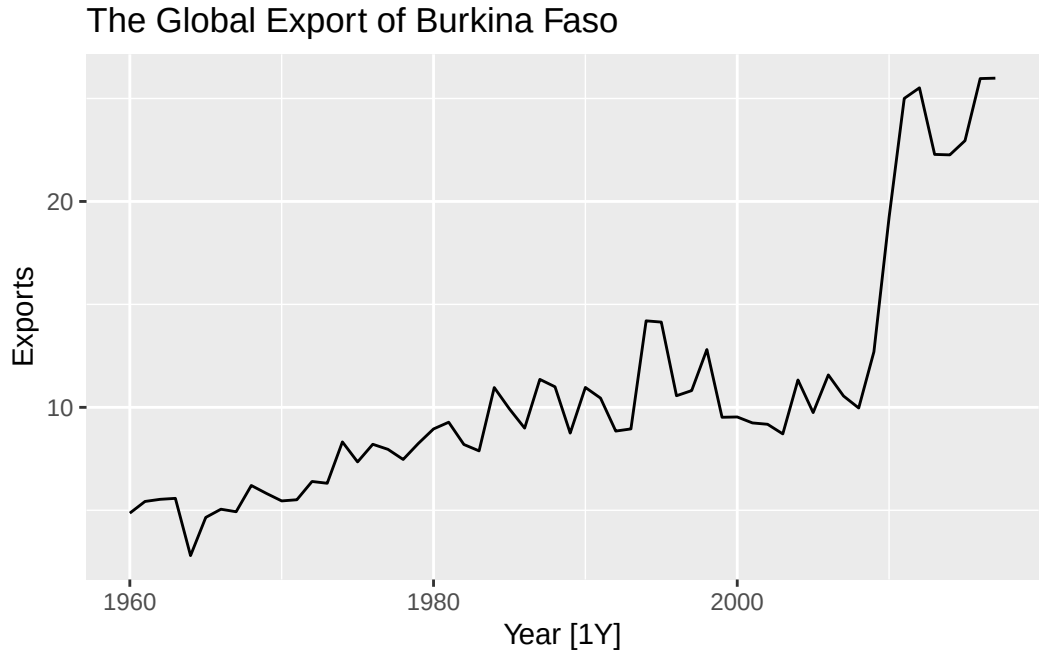
The first forecasted value for January 2019 is 95,186.56 pigs. Using the formula (\hat{y} \pm 1.96s), the manually calculated 95% prediction interval is [76,871.01, 113,502.10], while R produces [76,854.79, 113,518.30]. The two intervals are very close, with differences of approximately 16 pigs at each boundary. This suggests that the manual calculation is consistent with R's prediction interval. The interval indicates the range in which the actual number of pigs slaughtered in January 2019 is expected to fall, with 95% coverage under the model assumptions.

5. **Data set `global_economy` contains the annual Exports from many countries. Select one country to analyse.**

**a. Plot the Exports series and discuss the main features of the data.**

```
burkina_faso <- global_economy |>
  filter(Country == "Burkina Faso")


burkina_faso |> autoplot(Exports) + labs(title = "The Global Export of Burkina Faso")
```




**##Interpretation:**

Based on the plot, Burkina Faso's exports as a percentage of GDP show an overall upward trend, although there are periods of increases and decreases throughout the series. A noticeable increase can be observed from approximately 2009 onward, suggesting that exports became a larger share of the country's GDP. The fluctuations indicate variability in the series, while no seasonal pattern can be identified because the data are recorded annually.

**b. Use an ETS(A,N,N) model to forecast the series, and plot the forecasts.**

```
export_model <- burkina_faso |>
  model(ETS(Exports ~ error("A") + trend("N") + season("N") ))
```

```r
#Results
report(export_model)
```

```
Series: Exports
Model: ETS(A,N,N)
  Smoothing parameters:
    alpha = 0.9998999

  Initial states:
     l[0]
 4.796731

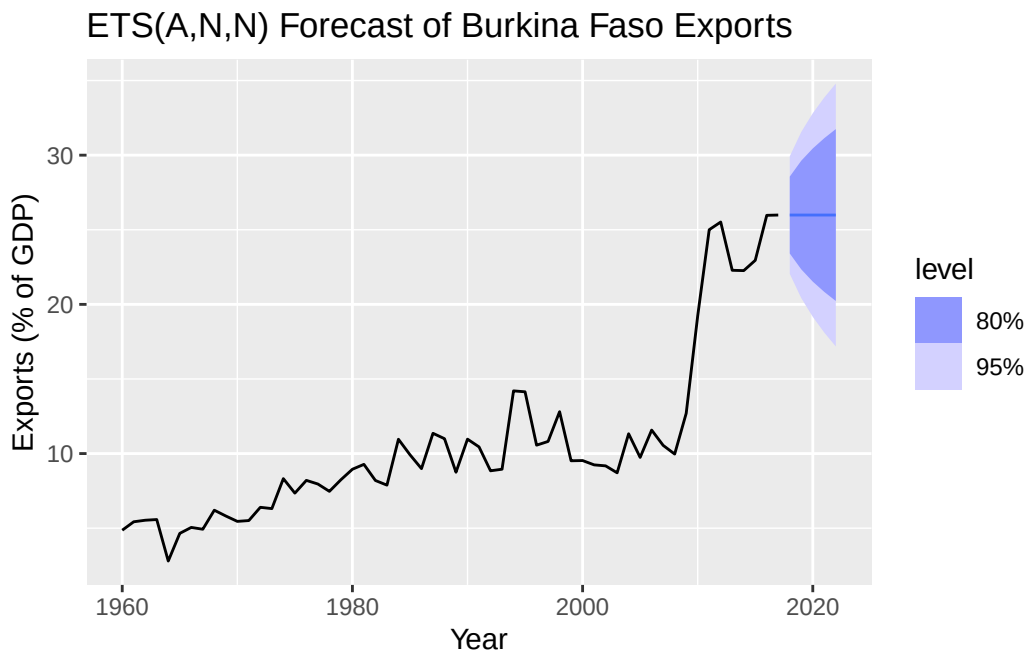
  sigma^2:  4.0402

     AIC     AICc      BIC
320.4557 320.9002 326.6371
```

```r
#Forecast
fc_burkina <- export_model |>
  forecast(h = 5)
fc_burkina
```

```
# A fable: 5 x 5 [1Y]
# Key:     Country, .model [1]
  Country      .model                                                      Year
  <fct>        <chr>                                                      <dbl>
1 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\")~  2018
2 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\")~  2019
3 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\")~  2020
4 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\")~  2021
5 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\")~  2022
# i 2 more variables: Exports <dist>, .mean <dbl>
```

```r
#Plot
fc_burkina |>
  autoplot(burkina_faso) +
  labs(
    title = "ETS(A,N,N) Forecast of Burkina Faso Exports",
    x = "Year",
    y = "Exports (% of GDP)"
  )
```




**##Interpretation:**

The ETS(A,N,N) model was used to forecast Burkina Faso's exports for the next five years. Since the model does not include a trend or seasonal component, the forecasted values remain constant across the forecast horizon. Although the historical data show an overall upward trend, the model assumes that future exports will remain around the most recently estimated level. The prediction intervals become wider as the forecast horizon increases, reflecting greater uncertainty in longer-term forecasts.

**c. Compute the RMSE values for the training data.**

```
export_model |>
  accuracy() |>
  select(.model, RMSE)
```

```
# A tibble: 1 x 2
  .model                                                      RMSE
  <chr>                                                      <dbl>
1 "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\"))"  1.98
```

```
glance(export_model) |>
  select(.model, AICc)
```

```
# A tibble: 1 x 2
  .model                                                              AICc
  <chr>                                                              <dbl>
1 "ETS(Exports ~ error(\"A\") + trend(\"N\") + season(\"N\"))"  321.
```

**##Interpretation:**

The RMSE measures the difference between the actual export values and the values fitted by the ETS(A,N,N) model. A lower RMSE indicates that the model fits the training data more closely. Since exports are measured as a percentage of GDP, the RMSE is expressed in percentage points.

**d. Compare the results to those from an ETS(A,A,N) model. (Remember that the trended model is using one more parameter than the simpler model.) Discuss the merits of the two forecasting methods for this data set.**

```
export_model2 <- burkina_faso |>
  model(ETS(Exports ~ error("A") + trend("A") + season("N") ))

#Results
report(export_model2)
```

```
Series: Exports
Model: ETS(A,A,N)
  Smoothing parameters:
    alpha = 0.9999
    beta  = 0.0001018731

  Initial states:
     l[0]      b[0]
 4.698716 0.3692187

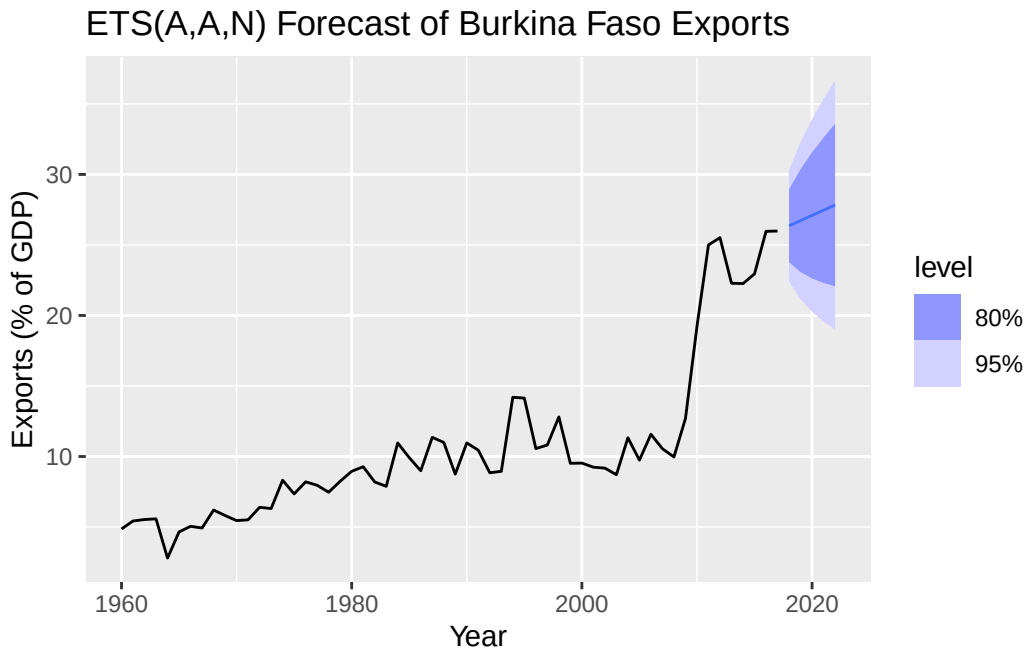
  sigma^2:  4.046

     AIC     AICc      BIC
322.4289 323.5827 332.7311
```

```
#Forecast
fc_burkina_holt <- export_model2 |>
  forecast(h = 5)
fc_burkina_holt
```

```
# A fable: 5 x 5 [1Y]
# Key:     Country, .model [1]
  Country      .model                                                      Year
  <fct>        <chr>                                                      <dbl>
1 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\")~  2018
2 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\")~  2019
3 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\")~  2020
4 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\")~  2021
5 Burkina Faso "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\")~  2022
# i 2 more variables: Exports <dist>, .mean <dbl>
```

```r
#Plot
fc_burkina_holt |>
  autoplot(burkina_faso) +
  labs(
    title = "ETS(A,A,N) Forecast of Burkina Faso Exports",
    x = "Year",
    y = "Exports (% of GDP)"
  )
```




```r
export_model2 |>
  accuracy() |>
  select(.model, RMSE)
```

```
# A tibble: 1 x 2
  .model                                                          RMSE
  <chr>                                                          <dbl>
1 "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\"))"  1.94
```

```
#AICc
glance(export_model2) |>
  select(.model, AICc)
```

```
# A tibble: 1 x 2
  .model                                                          AICc
  <chr>                                                          <dbl>
1 "ETS(Exports ~ error(\"A\") + trend(\"A\") + season(\"N\"))"  324.
```

**##Interpretation:**

Comparing the `ETS(A,N,N)` and `ETS(A,A,N)` models, we observe that `ETS(A,A,N)` has a lower training RMSE, indicating a better fit to the historical export data. However, ETS(A,N,N) has a lower AICc value of `320.9002` compared to `323.5827` for `ETS(A,A,N)`.

Although `ETS(A,A,N)` includes a trend component that can capture the upward movement in Burkina Faso's exports, it also uses an additional parameter, making the model more complex. The lower AICc of `ETS(A,N,N)` suggests that the improvement in fit from adding a trend does not sufficiently justify the additional complexity.

Therefore, based on AICc,`ETS(A,N,N)` is the preferred model for this dataset, even though `ETS(A,A,N)`has a lower training RMSE.

**e. Compare the forecasts from both methods. Which do you think is best?**

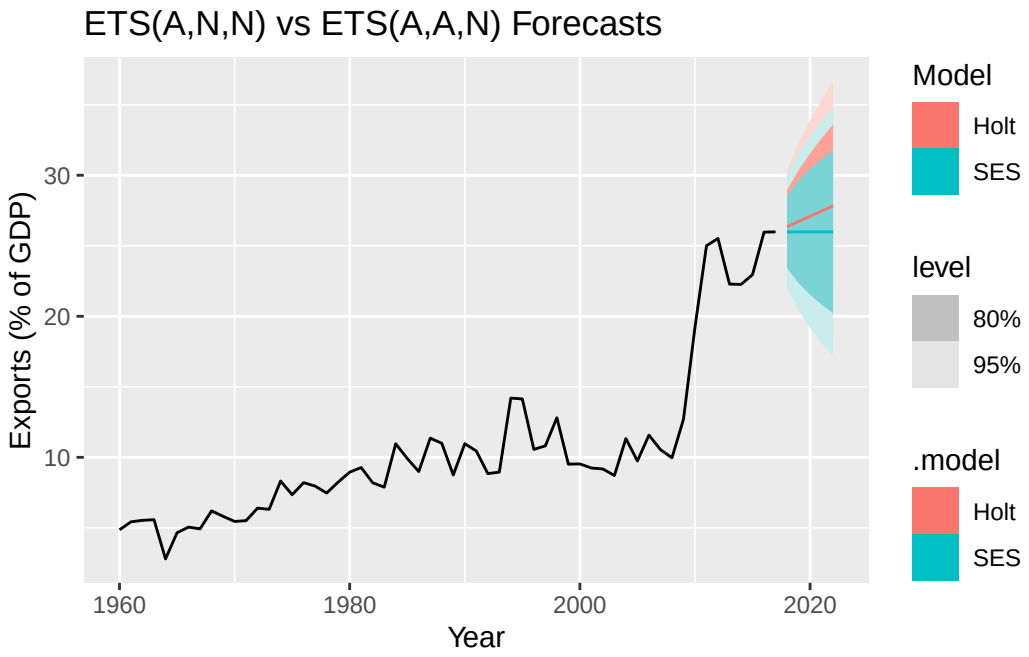
```
# Fit both models
export_comparison <- burkina_faso |>
  model(
    SES = ETS(Exports ~ error("A") +
                        trend("N") +
                        season("N")),

    Holt = ETS(Exports ~ error("A") +
                         trend("A") +
                         season("N"))
  )

# Forecast both models
```

```r
fc_comparison <- export_comparison |>
  forecast(h = 5)

# Plot
fc_comparison |>
  autoplot(burkina_faso) +
  labs(
    title = "ETS(A,N,N) vs ETS(A,A,N) Forecasts",
    x = "Year",
    y = "Exports (% of GDP)",
    colour = "Model"
  )
```




**##Interpretation:**

Comparing the forecasts from both models, ETS(A,N,N) produces constant forecasts because it does not include a trend component, while ETS(A,A,N) produces forecasts that follow an estimated linear trend.

Although ETS(A,A,N) has a lower training RMSE, ETS(A,N,N) has a lower AICc value of 320.9002 compared to 323.5827 for ETS(A,A,N). This suggests that the additional trend parameter does not provide enough improvement to justify the increased model complexity.

Therefore, I would prefer ETS(A,N,N) for this data set based on its lower AICc and simpler structure. However, evaluating the forecasts against future observations would provide stronger evidence of which model performs better.

**f. Calculate a 95% prediction interval for the first forecast for each model, using the RMSE values and assuming normal errors. Compare your intervals with those produced using R.**

```
# First forecast from each model
forecast_ses <- fc_burkina |>
  as_tibble() |>
  pull(.mean) |>
  first()

forecast_holt <- fc_burkina_holt |>
  as_tibble() |>
  pull(.mean) |>
  first()

# Extract RMSE for both models
rmse_ses <- export_model |>
  accuracy() |>
  pull(RMSE)

rmse_holt <- export_model2 |>
  accuracy() |>
  pull(RMSE)

# Calculate manual 95% prediction intervals
lower_ses <- forecast_ses - 1.96 * rmse_ses
upper_ses <- forecast_ses + 1.96 * rmse_ses

lower_holt <- forecast_holt - 1.96 * rmse_holt
upper_holt <- forecast_holt + 1.96 * rmse_holt

# Display manual intervals
tibble(
  Model = c("ETS(A,N,N)", "ETS(A,A,N)"),
  Lower = c(lower_ses, lower_holt),
  Upper = c(upper_ses, upper_holt)
)
```

```
# A tibble: 2 x 3
```

```
  Model      Lower Upper
  <chr>      <dbl> <dbl>
1 ETS(A,N,N)  22.1  29.9
2 ETS(A,A,N)  22.6  30.2
```

```r
# R's 95% prediction intervals
fc_comparison |>
  hilo(level = 95) |>
  as_tibble()
```

```
# A tibble: 10 x 6
   Country      .model  Year
   <fct>        <chr>  <dbl>
 1 Burkina Faso SES     2018
 2 Burkina Faso SES     2019
 3 Burkina Faso SES     2020
 4 Burkina Faso SES     2021
 5 Burkina Faso SES     2022
 6 Burkina Faso Holt    2018
 7 Burkina Faso Holt    2019
 8 Burkina Faso Holt    2020
 9 Burkina Faso Holt    2021
10 Burkina Faso Holt    2022
# i 3 more variables: Exports <dist>, .mean <dbl>, `95%` <hilo>
```

```r
# Extract RMSE for each model
rmse_values <- export_comparison |>
  accuracy() |>
  select(.model, RMSE)

# Extract the first forecast from each model
first_forecasts <- fc_comparison |>
  as_tibble() |>
  group_by(.model) |>
  slice_min(Year, n = 1) |>
  ungroup() |>
  select(.model, Year, .mean)

# Calculate manual 95% prediction intervals
manual_intervals <- first_forecasts |>
  left_join(rmse_values, by = ".model") |>
  mutate(
```

```
    Lower = .mean - 1.96 * RMSE,
    Upper = .mean + 1.96 * RMSE
  )

manual_intervals
```

```
# A tibble: 2 x 6
  .model  Year .mean  RMSE Lower Upper
  <chr>  <dbl> <dbl> <dbl> <dbl> <dbl>
1 Holt    2018  26.4  1.94  22.6  30.2
2 SES     2018  26.0  1.98  22.1  29.9
```

**##Interpretation:**

The first forecast year is 2018. The ETS(A,N,N) model predicts Burkina Faso's exports to be 25.98657% of GDP, with a 95% prediction interval of [22.04699, 29.92616]. The ETS(A,A,N) model predicts 26.35582% of GDP, with a 95% prediction interval of [22.41343, 30.29821].

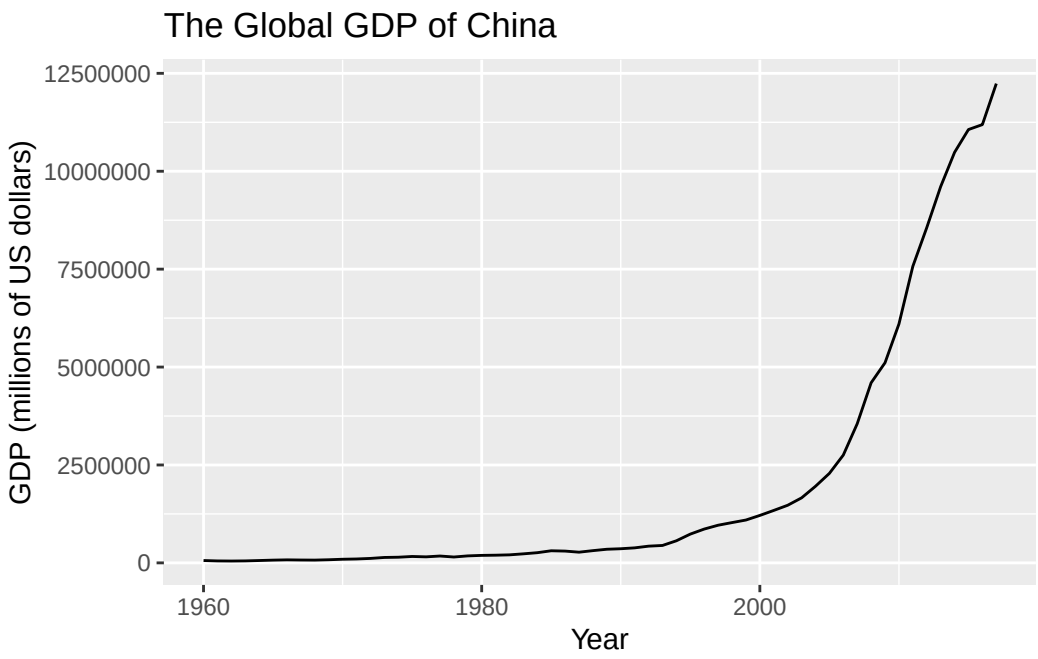
Both models produce similar prediction interval widths for the first forecast year, although the ETS(A,A,N) model predicts a slightly higher export percentage. The intervals become wider over the following years, reflecting increasing uncertainty in longer-term forecasts.

The manually calculated intervals using RMSE can be compared with these R-generated intervals to evaluate how closely the normal approximation matches the model-based results.

6. **Forecast the Chinese GDP from the `global_economy` data set using an ETS model. Experiment with the various options in the `ETS()` function to see how much the forecasts change with damped trend, or with a Box-Cox transformation. Try to develop an intuition of what each is doing to the forecasts.**

```
china_gdp <- global_economy |>
  filter(Country == "China")  |>
  mutate(GDP_million = GDP / 1000000)


china_gdp |> autoplot(GDP_million) + labs(title = "The Global GDP of China",
                                    x = "Year",
                                    y = "GDP (millions of US dollars)")
```




```
# Fit ETS models
gdp_modelc <- china_gdp |>
  model(
    ANN = ETS(GDP_million ~
                error("A") + trend("N") + season("N")),

    AAN = ETS(GDP_million ~
                error("A") + trend("A") + season("N")),

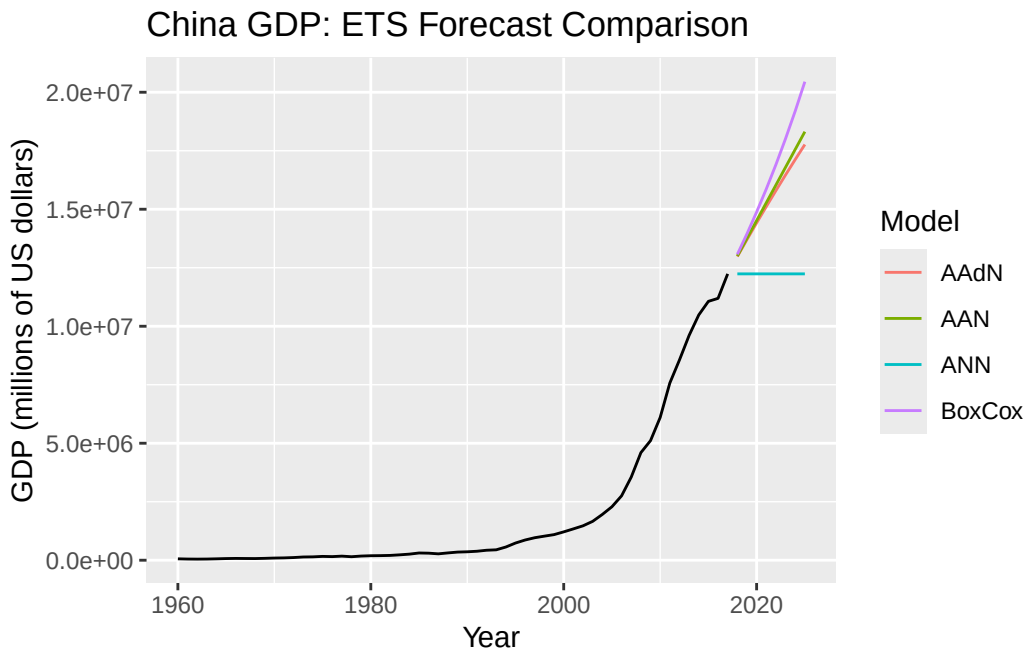
    AAdN = ETS(GDP_million ~
                 error("A") + trend("Ad") + season("N")),

    BoxCox = ETS(box_cox(GDP_million, 0.3) ~
                   error("A") + trend("A") + season("N"))
  )

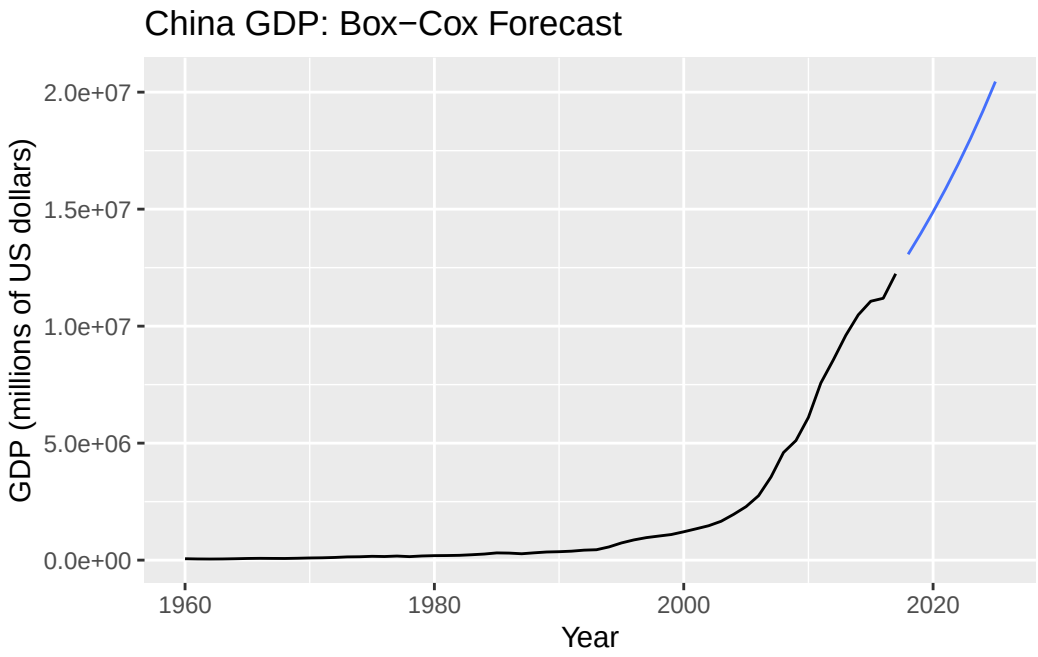
# Generate forecasts
fc_china <- gdp_modelc |>
  forecast(h = 8)

# Plot forecasts
fc_china |>
  autoplot(china_gdp, level = NULL) +
```

```
  labs(
    title = "China GDP: ETS Forecast Comparison",
    x = "Year",
    y = "GDP (millions of US dollars)",
    colour = "Model"
  )
```




```
fc_china |>
  filter(.model == "BoxCox") |>
  autoplot(china_gdp, level = NULL) +
  labs(
    title = "China GDP: Box-Cox Forecast",
    x = "Year",
    y = "GDP (millions of US dollars)"
  )
```




```
#Results
report(gdp_modelc)
```

```
Warning in report.mdl_df(gdp_modelc): Model reporting is only supported for
individual models, so a glance will be shown. To see the report for a specific
model, use `select()` and `filter()` to identify a single model.
```

```
# A tibble: 4 x 10
  Country .model  sigma2 log_lik   AIC  AICc   BIC     MSE    AMSE       MAE
  <fct>   <chr>    <dbl>   <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>     <dbl>
1 China   ANN    1.79e11   -868. 1742. 1743. 1748. 1.73e11 7.22e11 213215.
2 China   AAN    3.88e10   -823. 1655. 1657. 1666. 3.61e10 1.31e11  95917.
3 China   AAdN   3.96e10   -823. 1658. 1659. 1670. 3.62e10 1.29e11  94848.
4 China   BoxCox 2.33e 1   -207.  424.  425.  434. 2.17e 1 8.39e 1      3.82
```

```
gdp_modelc |>
  accuracy() |>
  select(.model, RMSE)
```

```
# A tibble: 4 x 2
  .model    RMSE
```

```
  <chr>    <dbl>
1 ANN    415727.
2 AAN    189990.
3 AAdN   190205.
4 BoxCox 206411.
```

```
glance(gdp_modelc) |>
  select(.model, AICc)
```

```
# A tibble: 4 x 2
  .model  AICc
  <chr>  <dbl>
1 ANN    1743.
2 AAN    1657.
3 AAdN   1659.
4 BoxCox  425.
```

**##Interpretation:**

The different ETS models were used to forecast China's GDP for the next eight years. The ETS(A,N,N) model produces constant forecasts because it does not include a trend component, while ETS(A,A,N) produces forecasts that continue along an upward trend.

The ETS(A,Ad,N) model includes a damped trend, which gradually reduces the rate of increase in the forecasts over time. This means the predicted GDP may continue increasing, but more slowly than with the ETS(A,A,N) model.

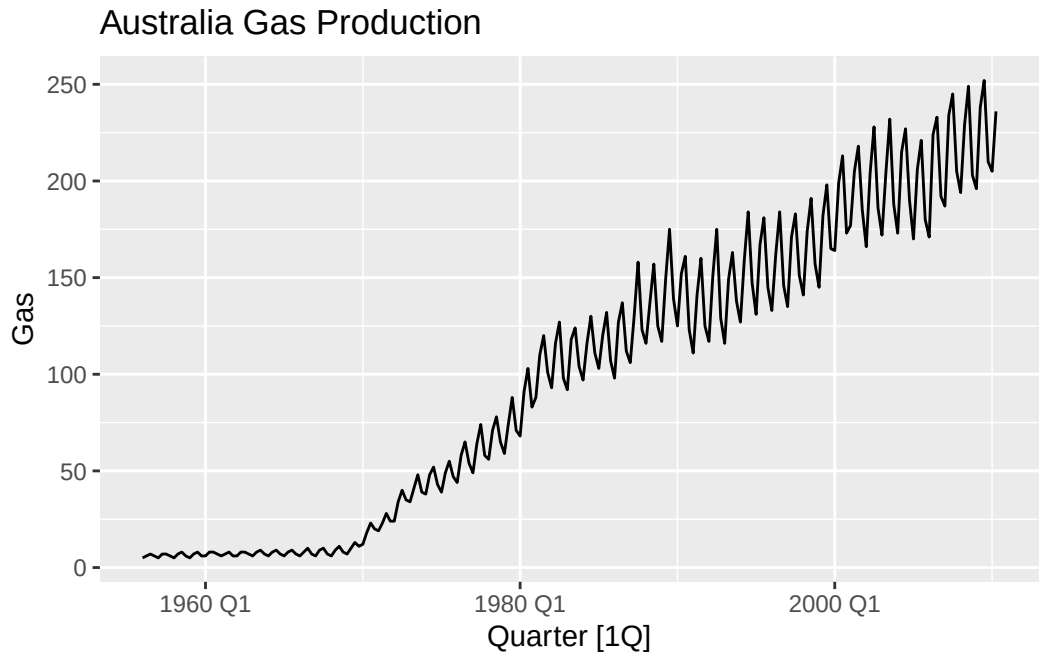
Based on the RMSE and AICc values obtained from the three untransformed models, ETS(A,A,N) appears to provide the best fit to the historical data because it has the lowest values.

The Box-Cox transformation changes the scale used to fit the model and can affect the shape of the forecasts after they are transformed back to the original GDP scale. Comparing its forecast with ETS(A,A,N) helps us understand how the transformation influences the predicted values.

Overall, ETS(A,A,N) appears preferable based on the reported training results, although future forecast accuracy would need to be evaluated using observations outside the training data.

7. **Find an ETS model for the Gas data from `aus_production` and forecast the next few years. Why is multiplicative seasonality necessary here? Experiment with making the trend damped. Does it improve the forecasts?**

```
aus_product <- aus_production |>
  select(Gas)
aus_product |> autoplot(Gas) + labs(title = "Australia Gas Production")
```




```
# Fit ETS models
aus_model_p <- aus_product |>
  model(
    additive = ETS(Gas ~ error("A") + trend("A") + season("A")),
    multiplicative = ETS(Gas ~ error("M") + trend("A") + season("M")),

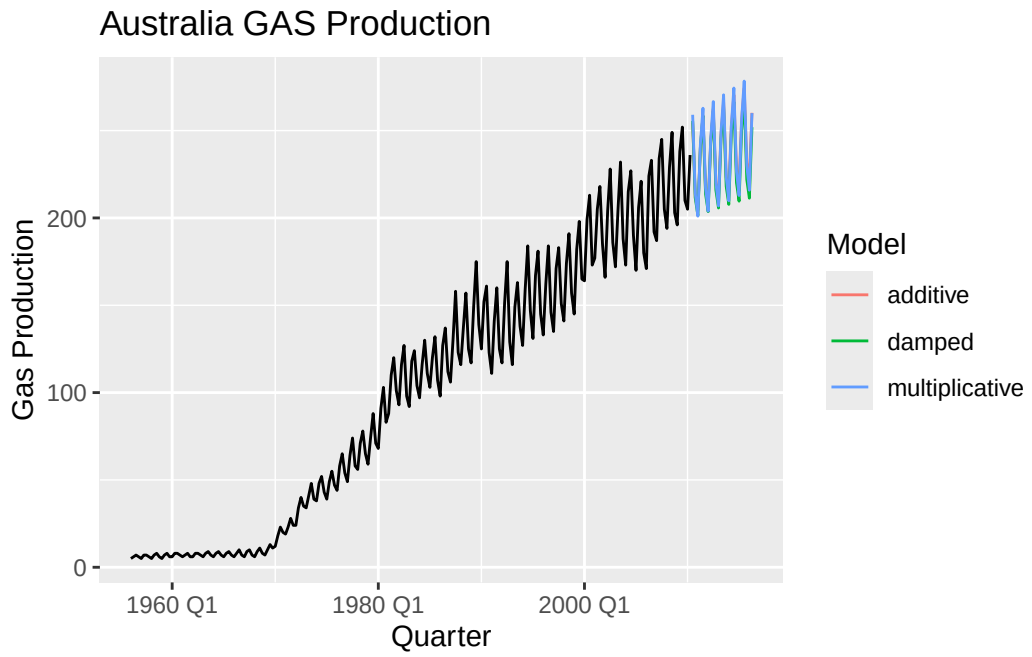
    damped = ETS(Gas ~ error("A") + trend("Ad") + season("M"))


  )
aus_model_p
```

```
# A mable: 1 x 3
      additive multiplicative        damped
       <model>        <model>       <model>
1 <ETS(A,A,A)>   <ETS(M,A,M)> <ETS(A,Ad,M)>
```

```
#Forcaste
fc_gas <- aus_model_p |>
  forecast(h = 24)
```

```r
# Plot forecasts
fc_gas |>
  autoplot(aus_product, level = NULL) +
  labs(
    title = "Australia GAS Production",
    x = "Quarter",
    y = "Gas Production",
    colour = "Model"
  )
```




```r
# Training accuracy
aus_model_p |>
  accuracy() |>
  select(.model, RMSE, MAE)
```

```
# A tibble: 3 x 3
  .model          RMSE   MAE
  <chr>          <dbl> <dbl>
1 additive        4.76  3.35
2 multiplicative  4.60  3.02
3 damped          4.22  2.81
```

```r
# Model selection
aus_model_p |>
  glance() |>
  select(.model, AICc) |>
  arrange(AICc)
```

```
# A tibble: 3 x 2
  .model          AICc
  <chr>          <dbl>
1 multiplicative 1682.
2 damped         1822.
3 additive       1873.
```

**##Interpretation:**

The Australian gas production data show an upward trend and a strong seasonal pattern. Observing the plot, we can see that the seasonal fluctuations increase as gas production increases over time.

After comparing additive and multiplicative ETS models, multiplicative seasonality appears more appropriate because it allows the seasonal fluctuations to change proportionally with the level of gas production. In contrast, additive seasonality assumes that seasonal fluctuations remain approximately constant over time.

The damped trend model can help reduce the rate of increase in the long-term forecasts compared to the model without damping. By comparing their RMSE and AICc values, we can determine whether including a damped trend improves the model's fit.

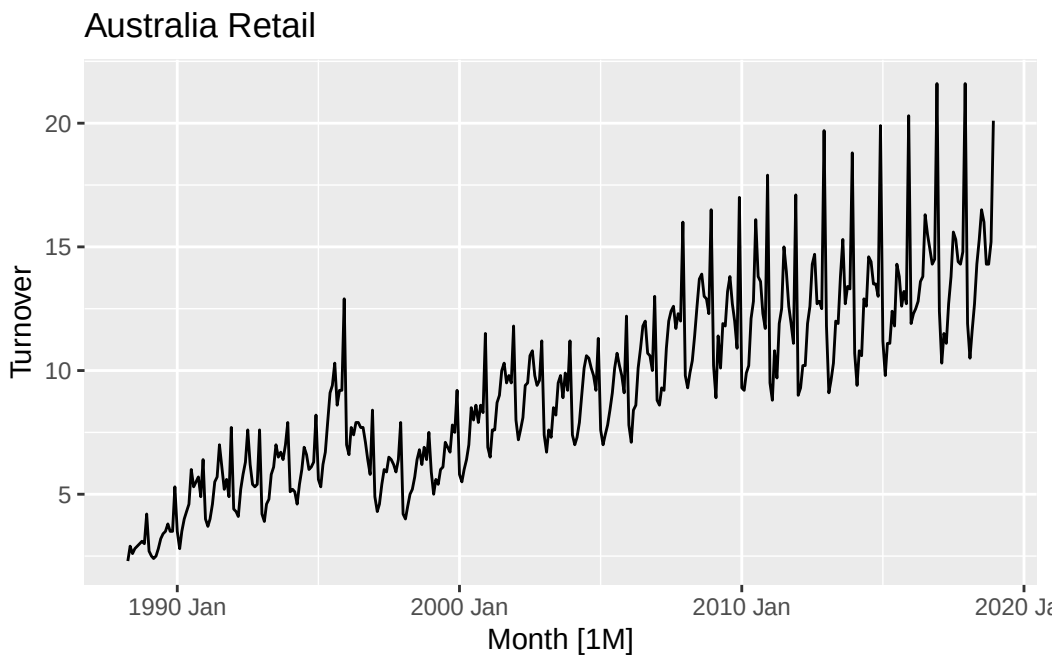
Overall, multiplicative seasonality appears suitable for this dataset because of the increasing seasonal variation. However, whether damping improves the forecasts depends on the model comparison results.

**8.8 Recall your retail time series data (from Exercise 7 in Section 2.10).**

a. **Why is multiplicative seasonality necessary for this series?**

```r
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

myseries |> autoplot(Turnover) + labs(title = "Australia Retail")
```




```
#Explore with a multiplicative seasonality

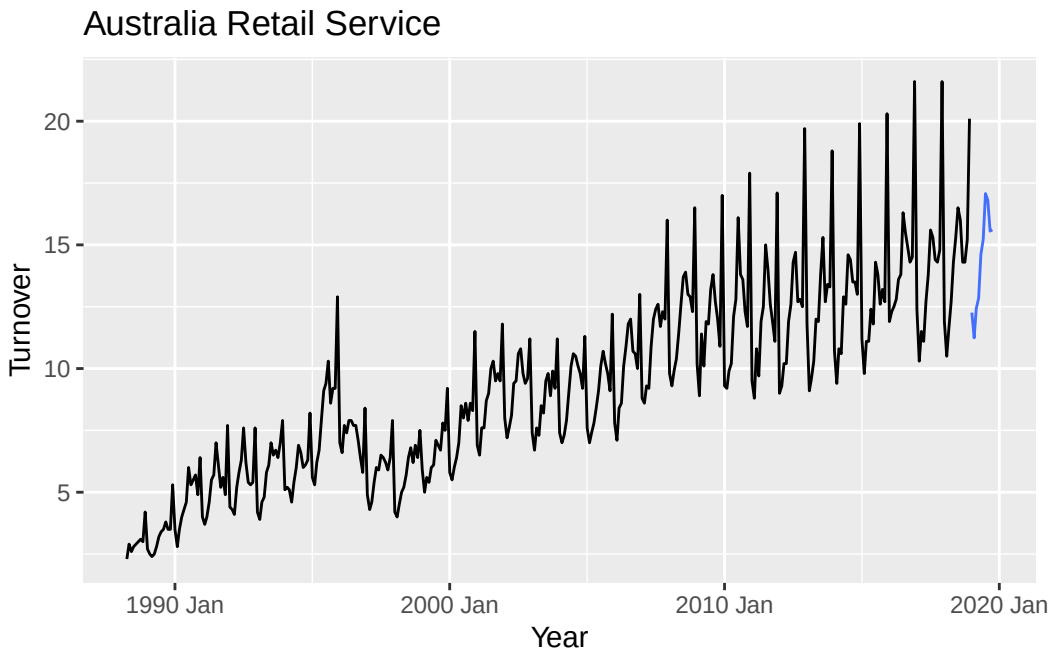
my_model <- myseries |>
  model(
    multiplicative = ETS(Turnover ~ error("M") + trend("A") + season("M"))

)

fc_series <- my_model |>
  forecast(h = 10)

# Plot forecasts
fc_series |>
  autoplot(myseries, level = NULL) +
  labs(
    title = "Australia Retail Service",
    x = "Year",
    y = "Turnover"

  )
```




**##Interpretation:**

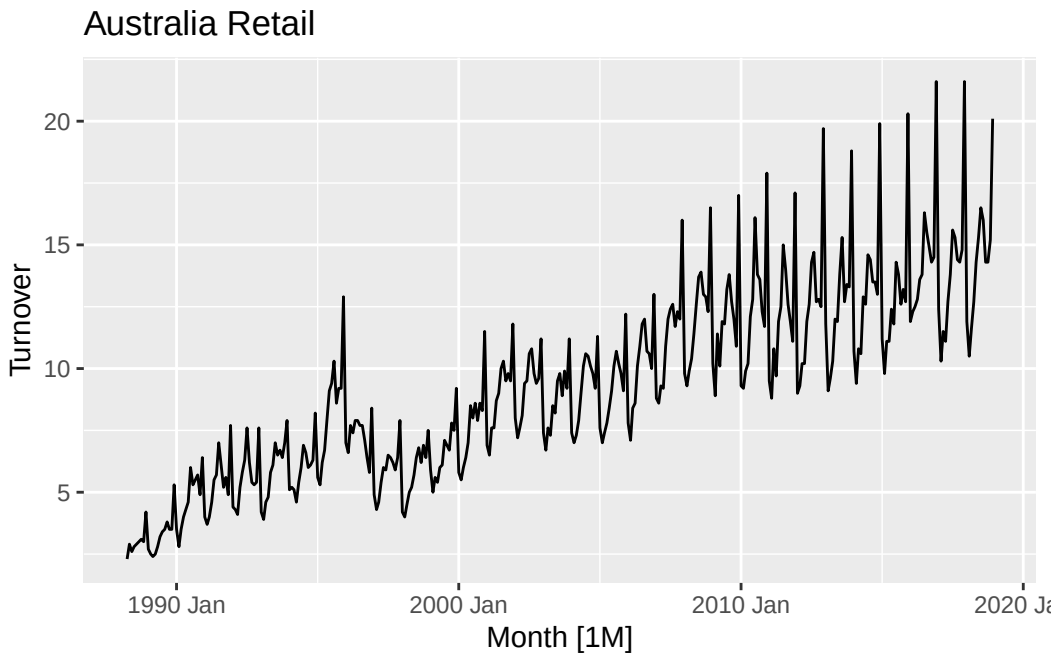
The retail turnover data show a seasonal pattern along with changes in the overall level of the series. As turnover increases, the seasonal fluctuations also appear to increase. This suggests that the seasonal variation is proportional to the level of turnover.

Therefore, multiplicative seasonality is more appropriate than additive seasonality because it allows the seasonal fluctuations to change with the level of the series.

**b. Apply Holt-Winters' multiplicative method to the data. Experiment with making the trend damped.**

```
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

myseries |> autoplot(Turnover) + labs(title = "Australia Retail")
```

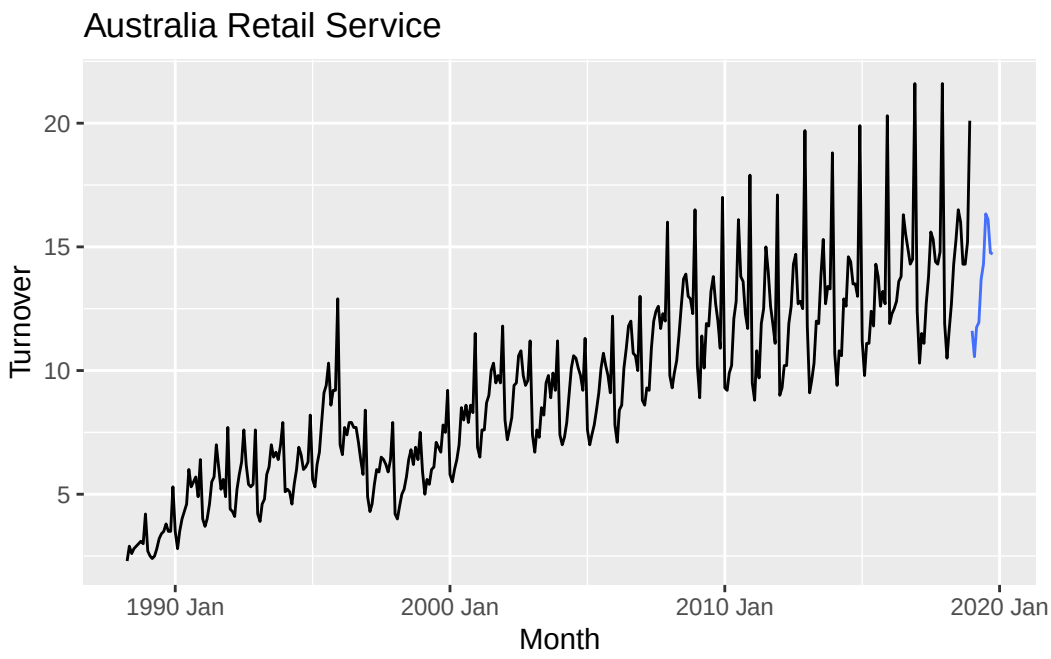



```
#Explore with a multiplicative seasonality

my_model2 <- myseries |>
  model(
    multiplicative = ETS(Turnover ~ error("M") + trend("Ad") + season("M"))
)
fc_series <- my_model2 |>
  forecast(h = 10)

# Plot forecasts
fc_series |>
  autoplot(myseries, level = NULL) +
  labs(
    title = "Australia Retail Service",
    x = "Month",
    y = "Turnover"

  )
```




```
# Training accuracy
my_model2 |>
  accuracy() |>
  select(.model, RMSE, MAE)
```

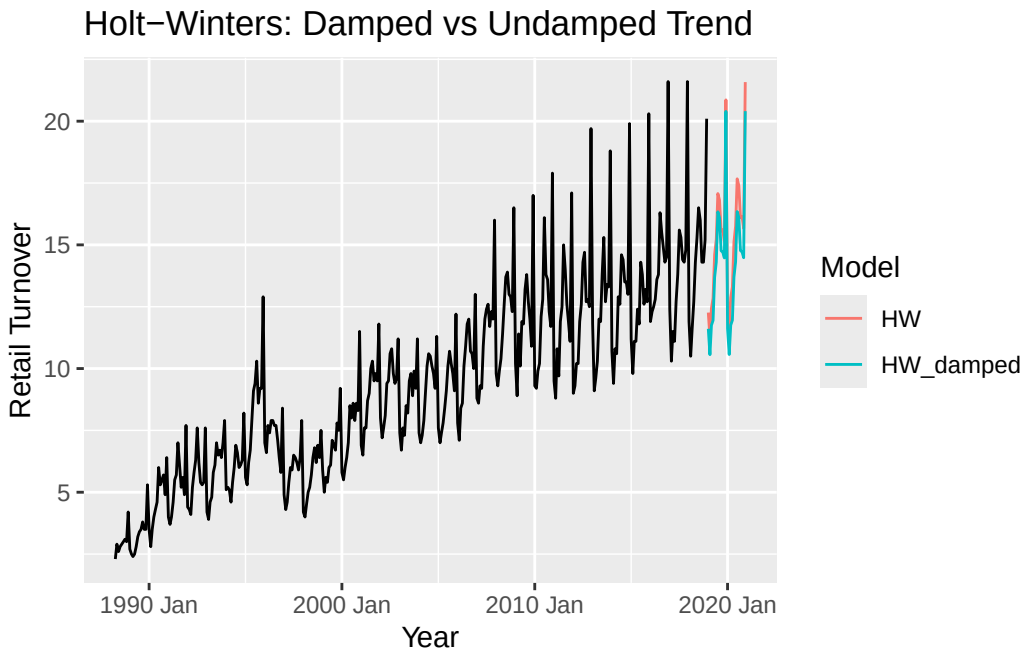
```
# A tibble: 1 x 3
  .model          RMSE   MAE
  <chr>          <dbl> <dbl>
1 multiplicative 0.616 0.444
```

```
# Model selection
my_model2 |>
  glance() |>
  select(.model, AICc) |>
  arrange(AICc)
```

```
# A tibble: 1 x 2
  .model          AICc
  <chr>          <dbl>
1 multiplicative 1783.
```

```r
# Compare Holt-Winters multiplicative models
my_model_comparison <- myseries |>
  model(
    # Without damping
    HW = ETS(Turnover ~
               error("M") +
               trend("A") +
               season("M")),

    # With damping
    HW_damped = ETS(Turnover ~
                      error("M") +
                      trend("Ad") +
                      season("M"))
  )
my_model_comparison |>
  forecast(h = 24) |>
  autoplot(myseries, level = NULL) +
  labs(
    title = "Holt-Winters: Damped vs Undamped Trend",
    x = "Year",
    y = "Retail Turnover",
    colour = "Model"
  )
```




**##Interpretation:**

Applying Holt- Winter multiplicative method we get a raisonable prediction with an upward trend.

**c. Compare the RMSE of the one-step forecasts from the two methods. Which do you prefer?**

```
# Compare RMSE
my_model_comparison |>
  accuracy() |>
  select(.model, RMSE) |>
  arrange(RMSE)
```

```
# A tibble: 2 x 2
  .model     RMSE
  <chr>     <dbl>
1 HW        0.613
2 HW_damped 0.616
```

**##Interpretation:**

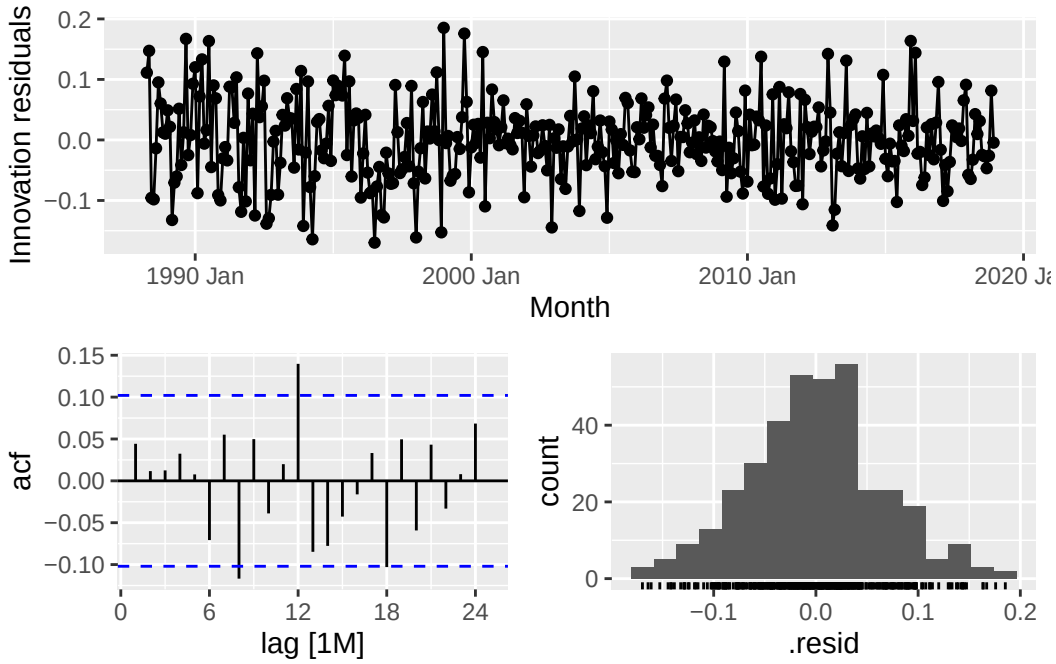
Comparing the RMSE values of the two Holt-Winters multiplicative models, the model without damping has an RMSE of 0.6130408, while the damped model has an RMSE of 0.6155067.

The model without damping has a slightly lower RMSE, indicating that its one-step-ahead forecasts are more accurate on the training data. However, the difference between the two RMSE values is very small, suggesting that adding a damped trend does not significantly improve the model's forecasting accuracy.

Therefore, I would prefer the Holt-Winters multiplicative model without damping, ETS(M,A,M), based on its lower RMSE.

**d. Check that the residuals from the best method look like white noise.**

```
my_model_comparison |>
  select(HW) |>
  gg_tsresiduals()
```




```
my_model_comparison |>
  select(HW) |>
  augment() |>
  features(.innov, ljung_box, lag = 24, dof = 0)
```

```
# A tibble: 1 x 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 HW        34.5    0.0765
```

**##Interpretation:**

Observing the residual diagnostic plots for the Holt-Winters multiplicative model, the residuals fluctuate around zero without a clear trend or seasonal pattern. The histogram shows an approximately bell-shaped distribution.

The ACF plot shows that most autocorrelation values remain within the significance bounds, although a few spikes appear around lags 8 and 12.

To further investigate whether the residuals behave like white noise, I performed a Ljung-Box test. The test produced a p-value of 0.0765, which is greater than the significance level of 0.05.

Therefore, we fail to reject the null hypothesis of no autocorrelation through the tested lags. Based on the residual plots and the Ljung-Box test, the residuals appear reasonably consistent with white noise, although some individual autocorrelation spikes remain.

**e. Now find the test set RMSE, while training the model to the end of 2010. Can you beat the seasonal naïve approach from Exercise 7 in Section 5.11?**

```
# Training data 2010
myseries_train <- myseries |>
  filter(year(Month) <= 2010)

# Test data from 2011--
myseries_test <- myseries |>
  filter(year(Month) > 2010)


fit <- myseries_train |>
  model(
    HW = ETS(Turnover ~
               error("M") +
               trend("A") +
               season("M")),

    SNAIVE = SNAIVE(Turnover)
  )

fc_test <- fit |>
  forecast(new_data = myseries_test)

fc_test |>
  accuracy(myseries_test) |>
```

```
  select(.model, RMSE, MAE) |>
  arrange(RMSE)
```

```
# A tibble: 2 x 3
  .model  RMSE   MAE
  <chr>  <dbl> <dbl>
1 SNAIVE  1.55  1.24
2 HW      1.78  1.60
```

**##Interpretation:**

To evaluate the forecasting accuracy, I trained the Holt-Winters multiplicative model and the seasonal naïve model using retail turnover data through December 2010. I then forecasted the test period beginning in January 2011.

The results show that the seasonal naïve model has a lower RMSE of 1.551981 compared to 1.782947 for the Holt-Winters model. The seasonal naïve model also has a lower MAE of 1.240625 compared to 1.597517 for Holt-Winters.

Therefore, the seasonal naïve model performs better on the test data. Although Holt-Winters captures both trend and seasonality, it does not outperform the simpler seasonal naïve approach for this dataset.

9. **For the same retail data, try an STL decomposition applied to the Box-Cox transformed series, followed by ETS on the seasonally adjusted data. How does that compare with your best previous forecasts on the test set?**

```
myseries_train <- myseries |>
  filter(year(Month) <= 2010)

myseries_test <- myseries |>
  filter(year(Month) > 2010)

fit_comparison <- myseries_train |>
  model(
    STL_ETS = decomposition_model(
      STL(box_cox(Turnover, 0.3) ~ season(window = "periodic")),
      ETS(season_adjust ~ season("N"))
    ),

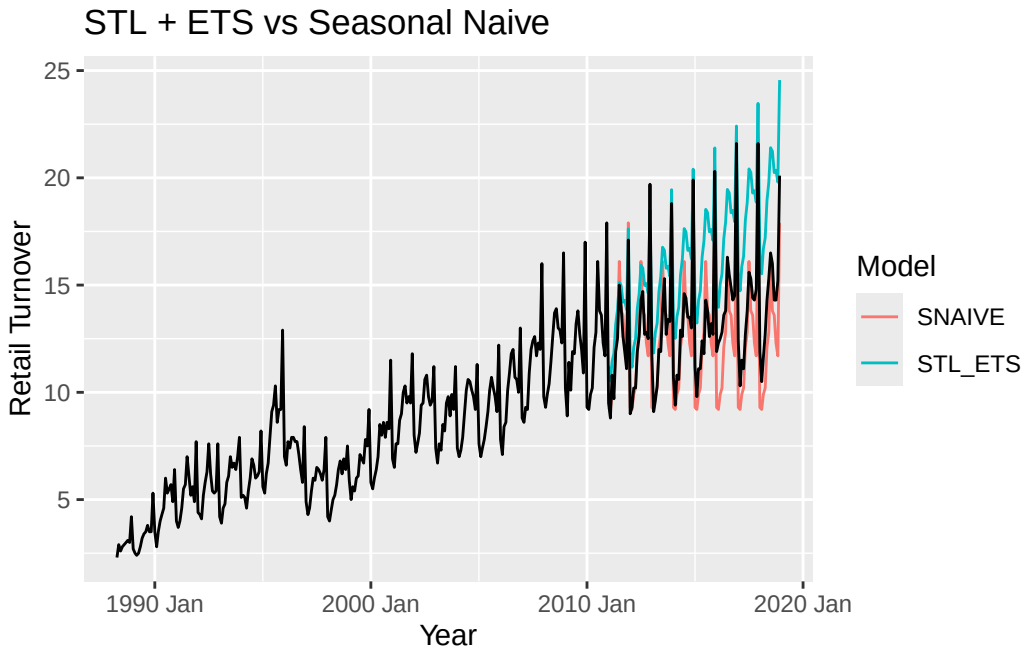
    SNAIVE = SNAIVE(Turnover)
  )
fc_comparison <- fit_comparison |>
```

```
  forecast(new_data = myseries_test)

fc_comparison |>
  accuracy(myseries_test) |>
  select(.model, RMSE, MAE) |>
  arrange(RMSE)
```

```
# A tibble: 2 x 3
  .model   RMSE   MAE
  <chr>   <dbl> <dbl>
1 SNAIVE   1.55  1.24
2 STL_ETS  3.41  3.11
```

```
fc_comparison |>
  autoplot(myseries, level = NULL) +
  labs(
    title = "STL + ETS vs Seasonal Naive",
    x = "Year",
    y = "Retail Turnover",
    colour = "Model"
  )
```



**##Interpretation:**

For this exercise, I applied an STL decomposition to the Box-Cox-transformed retail turnover series and then fitted an ETS model to the seasonally adjusted data. The model was trained using observations through December 2010, and forecasts were generated for the remaining test period.

Comparing the test RMSE values, the STL + ETS model has an RMSE of 3.411113, while the seasonal naïve model has a lower RMSE of 1.551981.

Observing the forecast plot, the STL + ETS model predicts a continuing upward trend, while the seasonal naïve model repeats the seasonal pattern from the last training year. The STL + ETS forecasts tend to overestimate the actual retail turnover, especially toward the end of the forecast period.

Therefore, the seasonal naïve model remains the preferred method for this dataset because it produces more accurate forecasts on the test data. The Box-Cox transformation and STL decomposition did not improve forecasting accuracy compared with the simpler seasonal naïve approach.